IA Hybride et Apprentissage Automatique pour les SHS · L6SOIAAU

CM 8 : Réseaux de neurones multicouches (MLP)

Ljudmila PETKOVIĆ

Semestre 2, 2025-2026
27 mars 2026

Sorbonne Université
Licence « Sciences du langage » (L3 ScLan)
UFR Sociologie et Informatique pour les Sciences Humaines

Cours adapté de Khichane (2025) et de Tavenard (s. d.)

LETTRES SORBONNE UNIVERSITÉ

Plan

1. *Deep Learning*
2. Du perceptron au MLP
3. Choisir l'architecture d'un MLP
4. Fonctions d'activation

LETTRES SORBONNE UNIVERSITÉ

Deep Learning

Pourquoi le *Deep Learning* (*DL*) s'est imposé

Le DL n'est pas nouveau : son essor repose sur trois facteurs :

- l'augmentation massive des données disponibles
- la puissance de calcul accrue, notamment grâce aux GPU
- la montée en complexité des tâches (vision, parole, texte) → multimodalité

Principe

Le succès du DL : rencontre entre **modèles**, **données** et **calcul**.

LETTRES SORBONNE UNIVERSITÉ

Les réseaux de neurones simples ne suffisent pas toujours

Les réseaux simples suffisent pour les tâches faciles.

Ils peinent à modéliser des régularités riches ou hiérarchiques.

- image : passer de motifs locaux à des formes complexes
- séquence : le sens dépend de relations entre éléments éloignés
- problèmes réels : sous-tâches traitées en parallèle

Conséquence

Besoin d'architectures capables de construire progressivement des **représentations plus utiles** que les données brutes d'origine.

LETTRES SORBONNE UNIVERSITÉ

Du réseau simple au réseau profond

DL : réseau comportant plusieurs couches cachées

$\rightarrow$ plusieurs transformations intermédiaires.

- une couche d'entrée **reçoit** les données
- une ou plusieurs couches cachées les **transforment**
- une couche de sortie produit la **prédiction finale**

« deep » : profondeur de la chaîne de transformations

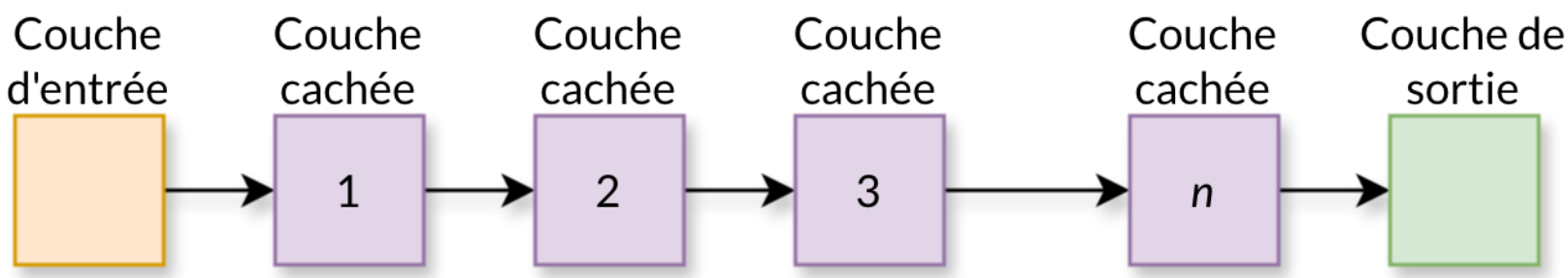


Figure 1 – Schéma d'un réseau de neurones DL, adapté de Khichane (2025).

MLP

Point de départ : le perceptron

Modèle

$$\hat{y} = \sum_{j=1}^{d} x_j w_j + b$$

- sortie prédite : combinaison **linéaire** des entrées + biais
- on optimise donc dans une famille de **modèles linéaires**
- famille simple, interprétable, mais **limitée en expressivité**

Objectif central : modèle plus complexe

Obtenir un modèle capable d'apprendre des relations **non linéaires**

LETTRES SORBONNE UNIVERSITÉ

Idée clé : empiler des couches

Multi-Layer Perceptron (*MLP*)

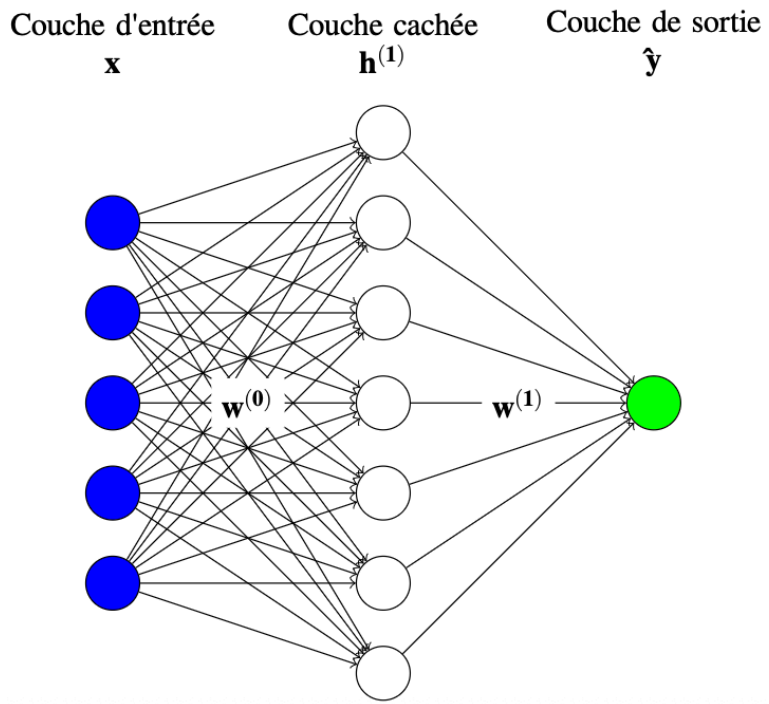


Figure 2 – Modèle d'une couche cachée, repris de Tavenard (s. d.).

Une couche cachée entre l'entrée et la sortie.

La couche cachée élargit-elle la famille des modèles ?

Théorème d'approximation universelle

Toute fonction continue définie sur un ensemble compact peut être approchée d'aussi près que l'on veut par un réseau neuronal à une couche cachée avec activation sigmoïde.

Une couche cachée peut approximer toute fonction continue.

Ce que le théorème ne dit pas :

- combien de neurones cachées sont nécessaires
- si les algorithmes d'optimisation convergeront vers la solution

En pratique

Plusieurs couches sont souvent plus efficaces qu'une seule.

LETTRES SORBONNE UNIVERSITÉ

Perceptron multicouches

Multi-Layer Perceptron (MLP)

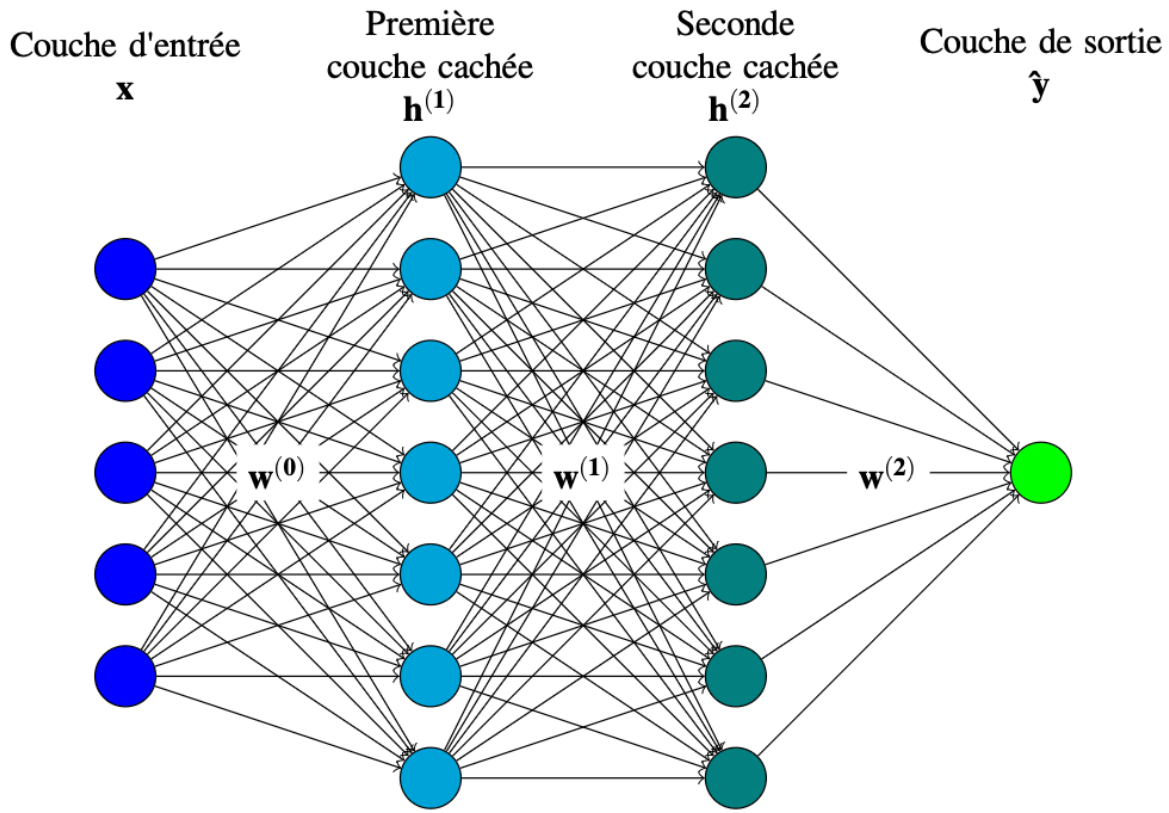


Figure 3 – Modèle de plusieurs couches cachées, repris de Tavenard (s. d.).

Définition d'un MLP

Perceptron multicouche (MLP)

Réseau de neurones pleinement connecté [a] composé :

- d'une couche d'entrée,
- d'**une ou plusieurs couches cachées**,
- d'une couche de sortie.

a. chaque neurone est relié à tous ceux de la couche suivante

- biais, même s'ils ne sont pas toujours représentés
- DL : dès qu'on empile plusieurs couches apprenables

Architecture

Qu'est-ce qui est fixé par le problème ?

Certaines dimensions d'un MLP sont imposées par la tâche :

- **nombre d'entrées** = nombre de variables descriptives
- **nombre de sorties** = nature de la cible à prédire

Ce qui reste à choisir est de l'ordre de l'**hyper-paramétrage** :

- nombre de couches cachées
- nombre de neurones par couche
- activation
- régularisation
- optimisation

LETTRES SORBONNE UNIVERSITÉ

Exemple 1 : jeu de données `iris`

Prédire la « cible » (**3 classes**) à partir de **4 attributs**

Attribut	SL	SW	PL	PW	cible
Type	réel	réel	réel	réel	classe

- attributs : `sepal length`, `sepal width`, `petal length`, `petal width`
- classes cibles : `setosa`, `versicolor`, `virginica`

Architecture

- Couche d'entrée : **4 neurones**
- Couche de sortie : **3 neurones**

LETTRES SORBONNE UNIVERSITÉ

Nombre de neurones de sortie selon la tâche

Type de tâche	**Sortie**	**Interprétation**
Régression	1 ou plus	Valeur(s) réelles à prédire
Classification binaire	1	Proba de la classe positive
Classification multi-classes	K	Une probabilité par classe

Erreur fréquente

Choisir la bonne taille de la couche de sortie ne suffit pas :

- il faut aussi choisir la bonne **activation de sortie**.

LETTRES SORBONNE UNIVERSITÉ

Fonctions d'activation

Pourquoi des fonctions d'activation ?

- sans activation, chaque couche reste **linéaire**
- plusieurs couches linéaires successives $\equiv$ **une seule**
- le réseau n'apprend donc que des relations **linéaires**
- les fonctions d'activation introduisent la **non-linéarité**
- elles permettent d'apprendre des fonctions **complexes**

LETTRES SORBONNE UNIVERSITÉ

Activation sigmoïde

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- sortie dans $[0, 1]$
- interprétation probabiliste utile en sortie binaire
- saturation forte aux extrêmes (la f-on se bloque)

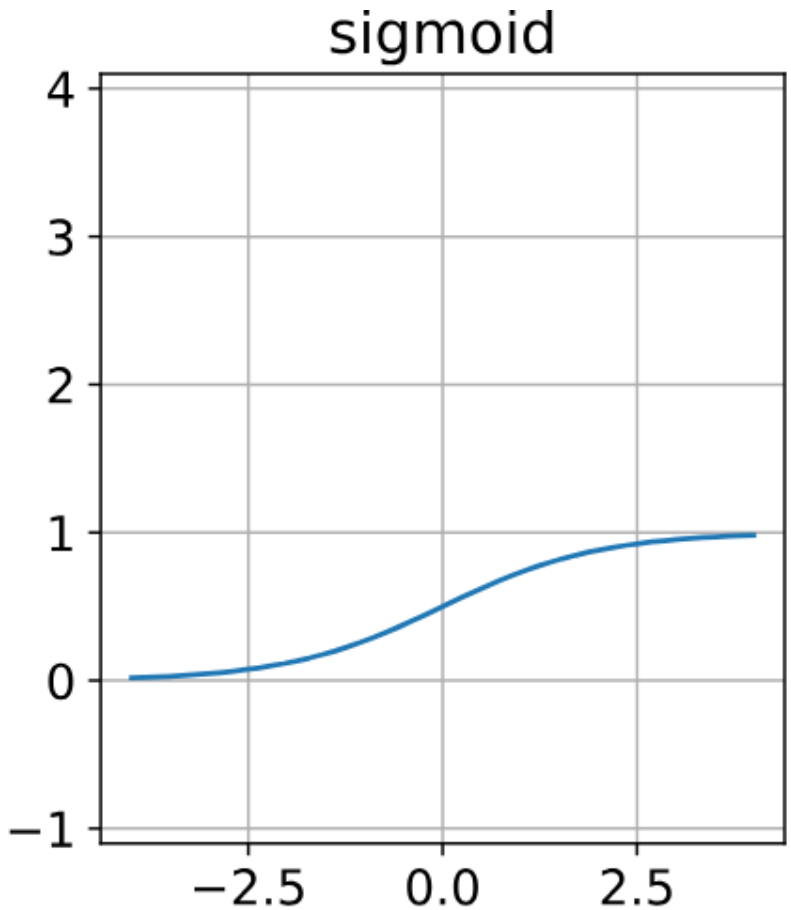


Figure 4 – Fonction sigmoïde, reprise de Tavenard (s. d.).

Activation tanh

$$\tanh(x) = \frac{2}{1 + e^{-2x}} - 1$$

- sortie dans $[-1, 1]$
- centrée en 0
- saturation pour les grandes valeurs positives/négatives (la f-on se bloque)

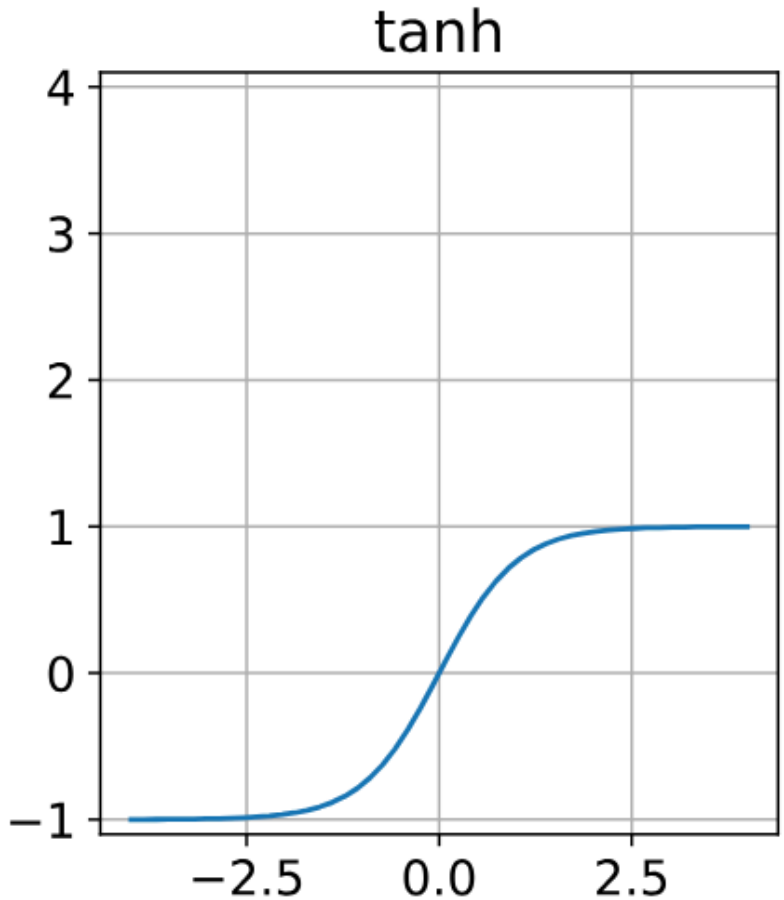


Figure 5 – Fonctions tanh, reprise de Tavenard (s. d.).

Activation ReLU

$$\mathrm{ReLU}(x) = \begin{cases} x & \text{si } x > 0 \\ 0 & \text{sinon} \end{cases}$$

- simple à calculer
- pas de saturation du côté positif
- très utilisée dans les couches cachées aujourd'hui

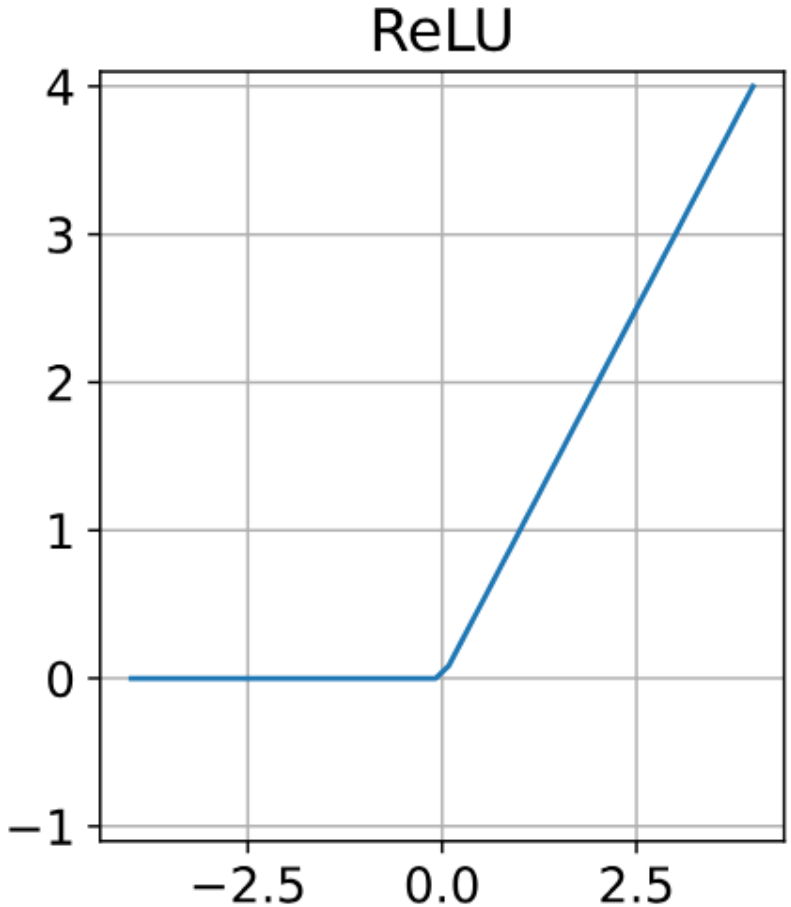


Figure 6 – Fonctions ReLU, reprise de Tavenard (s. d.).

Le cas particulier de la couche de sortie

- la couche de sortie a sa **propre f-on d'activation** : φ_{out}
- son choix dépend de la **tâche** (nature des valeurs à prédire)
- produire des valeurs cohérentes avec la sortie attendue

Exemples

- **Régression positive** (ex. prix de logements) : **ReLU**
- **Classification binaire** : **sigmoïde** $\rightarrow$ sortie dans $[0, 1]$
- **Classification multi-classes** : **softmax** $\rightarrow$ sortie dans $[0, 1]$

LETTRES SORBONNE UNIVERSITÉ

Sortie pour une tâche de régression

Si l'on prédit une quantité :

- sans contrainte : activation linéaire souvent suffisante
- **non négative** : ReLU peut être pertinente

Exemple

Dans le cas des prix immobiliers, une sortie négative est absurde.

Sortie pour une classification binaire

- un seul neurone de sortie
- probabilité associé à la classe positive
- valeur située dans l'intervalle $[0, 1]$
- la sigmoïde est le choix naturel

$$\hat{y} = \sigma(o) \in [0, 1]$$

o : valeur de neurone de sortie avant la f-on d'activation

Interprétation

$\hat{y}$ peut être lu comme la probabilité estimée de la classe positive.

LETTRES SORBONNE UNIVERSITÉ

Sortie pour une classification multi-classes

Si le problème a K classes :

- on met K neurones de sortie
- chacun correspond à une classe
- les sorties doivent être positives et sommer à 1

Fonction softmax

transforme un vecteur de scores en distribution de probabilités.

LETTRES SORBONNE UNIVERSITÉ

keras

TD 8 : implémentation avec `keras`

Les exercices dans le cadre du TD 8 sont disponibles à l'adresse
https://rtavenar.github.io/deep_book/fr/content/fr/mlp.html#declarer-un-mlp-en-keras

LETTRES SORBONNE UNIVERSITÉ

Références



Khichane, M. (juin 2025). ***Le Machine Learning et l'IA générative avec Python : de la théorie à la pratique***. 2e éd. https://www.editions-eni.fr/livre/le-machine-learning-et-l-ia-generative-avec-python-de-la-theorie-a-la-pratique-2e-edition-97824090500. Éditions ENI, p. 916. ISBN : 978-2-409-05001-5. URL : https://www.editions-eni.fr/livre/le-machine-learning-et-l-ia-generative-avec-python-de-la-theorie-a-la-pratique-2e-edition-9782409050015 (*voir pp. 1, 6*).



Tavenard, R. (s. d.). ***Perceptrons multicouches — Introduction au Deep Learning (notes de cours) — rtavenar.github.io***. https://rtavenar.github.io/deep_book/fr/content/fr/mlp.html. [Consulté le 26 mars 2026] (*voir pp. 1, 9, 11, 19-21*).

LETTRES SORBONNE UNIVERSITÉ

Licence

Le contenu de cette présentation est sous licence CC-BY-NC-SA 4.0
Utilisation non commerciale – Partage dans les mêmes conditions.



LETTRES SORBONNE UNIVERSITÉ